

CS 122A Custom Laboratory Project Report

Viper

Samantha Placito Melendrez, Erick Acuna, Laiba Nasir

June 10, 2026

Introduction

Our laser turret project, *Viper*, is a target-tracking system that uses a camera, an FPGA, and a microcontroller to detect a colored object in real time. When it locks onto a color object, it then aims a laser at it and briefly chirps a buzzer. The **OV7670** camera captures a 480x272 RGB565 video stream and feeds it into the FPGA. The FPGA then handles pixel capture and color blob detection, and drives the LCD screen. A Raspberry Pi Pico 2W reads the centroid coordinates from the FPGA over SPI before driving the servo motors to point the laser at the target. When a target is found, the laser turns on, but when it's lost, it turns off.

*** For this project, we certainly aimed high, and we did achieve some working parts of individual components, but unfortunately, we did not integrate everything like we wanted to. More details are below in the report.

Elements of Complexity

- Live camera to FPGA pixel pipeline
 - The camera model we're using is OV7670, and we have a camera output on an 8-bit bus. This was the most complex and time-consuming part of the project because of making the clock cycles in sync along with ensuring the RGB565 value is correctly

assembled. We went through several intervals of byte alignments since they were swapped several times.

- **Cross-checking the clock for the line buffering**

- The FPGA has a clock, and so does the LCD. During the midpoint demo, they were running at different frequencies and we did not understand how to connect them. Then, after a lot of debugging, we got a much better image [though still a little blurry] using a double buffer and synchronizer.

- **Camera → LCD resolution matching**

- Both the camera and LCD have different resolutions, and with the help of AI debugging, we were able to match the Camera frame to the LCD frame.

- **I2C communication with the Pico** so the Pico can configure the camera.

We want the camera's output format to be RGB565. The main purpose of this was to configure the camera register in RGB565 format, resolution, and timing. We implemented the configuration for the camera registers for the RGB565 format, resolution, and timing. But after implementing our configuration, we saw a color shift, and our color mapping was wrong. We spent several hours fixing this during finals week, but it did not work out.

- **SPI communication with the pico and FPGA** so the camera can detect a target color. We completed the Pico side SPI implementation, but not the FPGA one. Unfortunately, since the I2C color configuration was not correct, we could not integrate this into our project.

- **Implementation of 2 servo motors, a laser, and a buzzer** in a manner where the pico passes in the PWM and coordinates in this module so the servo can turn horizontally and vertically accordingly. [Since we did not complete the color detection, we programmed our servo with a pre-determined path such as a square, to show how it was communicating with the pico to get PWM and the coordinates, but unfortunately, it could not be done with the color detection feature] During the demo, we showed the movement of this system and explained that these features are working independently now, they were meant to coordinate by communicating with the Pico.

User Guide

For this project, you would need to make sure to follow the steps:

1. Power on your system. The moment it is on, the pico will begin the camera configuration immediately. It won't take that long.
2. Wait for initialization since the Pico doesn't start until the camera registers are written.
3. Hold a colored object in front of the camera. When you do this, the FPGA will compare each pixel against a RGB565 color threshold range. If the color is within that range, the tracking will begin.
4. Watch the turret move, and the laser turn on, and a buzzer chirp as it steers towards the colored object that was detected.
5. Move the object so that the turret can follow it in real time.

6. Remove or hide the object and observe how the laser turns off, and the turret stays in its last position while the camera waits for the colored object to appear again.

7. Watch how the LCD screen displays what the camera sees.

*** This was our intended behavior. We did not finish the color-detection code (due to time constraints during finals week), so the full end-to-end tracking does not run. We had communication wired up between the FPGA, LCD, and Pico, but it wasn't passing data correctly.

*** We do have working subsystems: a live camera image on the LCD, servo motion, and laser + buzzer functionality. Each of these works independently , when you power everything on, the camera picks up your image (blurry, with incorrect color mapping) and the servos, laser, and buzzer all operate. What's missing is the integration: the color-detection link that would tie what the camera sees to how the turret moves. So rather than a single integrated system, we have several functional pieces that still need that connecting layer.

Hardware Components Used

- Raspberry Pi Pico 2 W
- IceSugarPro FPGA
- OV7670 camera
- Breadboards
- Jumper cables
- 480x272 LCD screen

- PMOD tape to pin adapter
- KY-008 650nm Laser
 - NPN PN2222 transistor (amplifier)
- SG90 servos
 - Wall-plugged 5V power supply
- Active buzzer

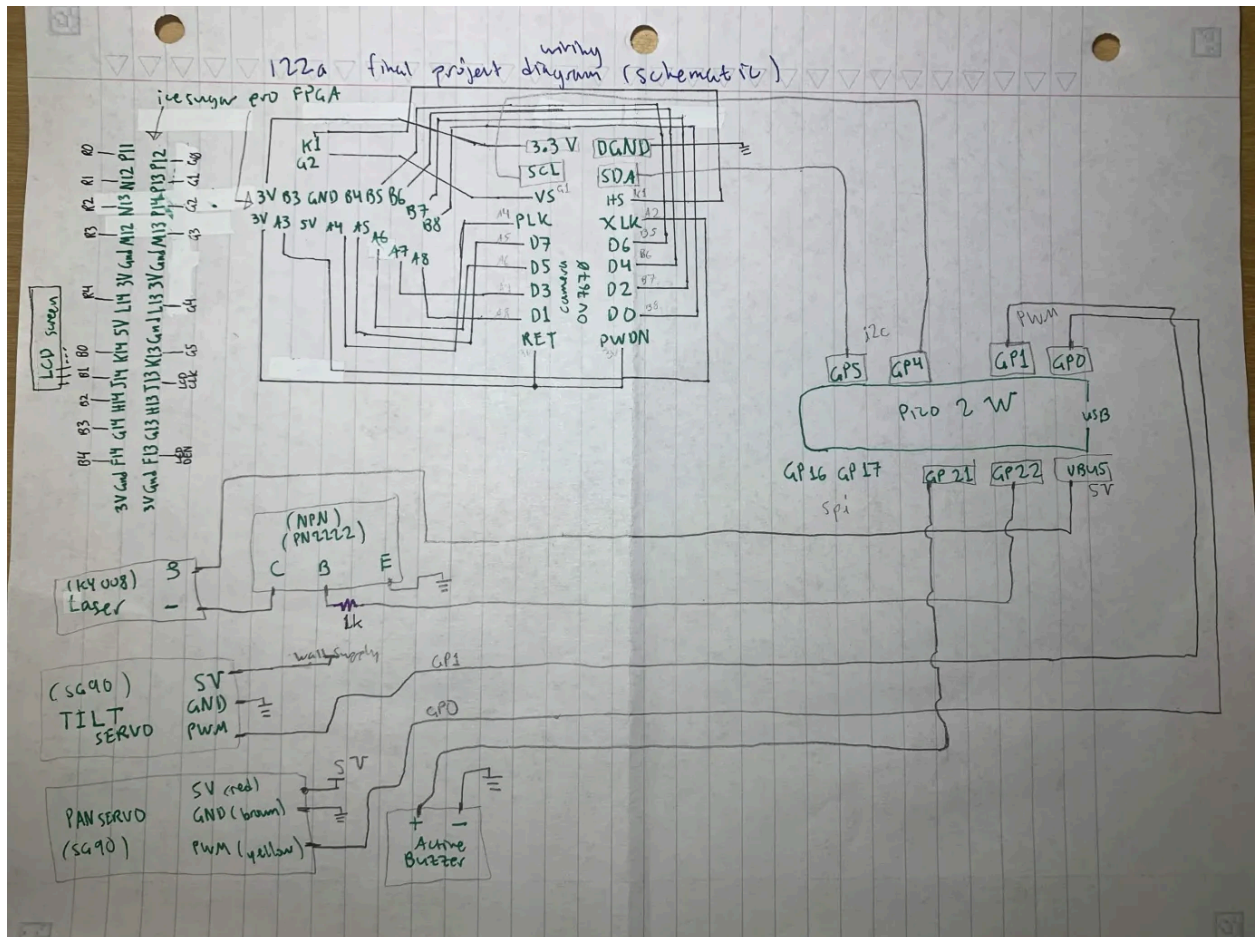
Protocols Used

- I2C
 - *We used the Raspberry Pi Pico I2C library to configure the OV7670 camera and set its output as RCG565 right at startup.*
- SPI
 - *We used SPI on the FPGA and Pico. The pico was the master while the FPGA was the slave. The pico sends 1 real byte to the FDPGA as a read request before the FPGA responds with 5 bytes while also reporting the centroid coordinates to the pico. We completed the Pico side SPI implementation, but not the FPGA one. Unfortunately, since the I2C color configuration was not correct, we could not integrate this into our project.*

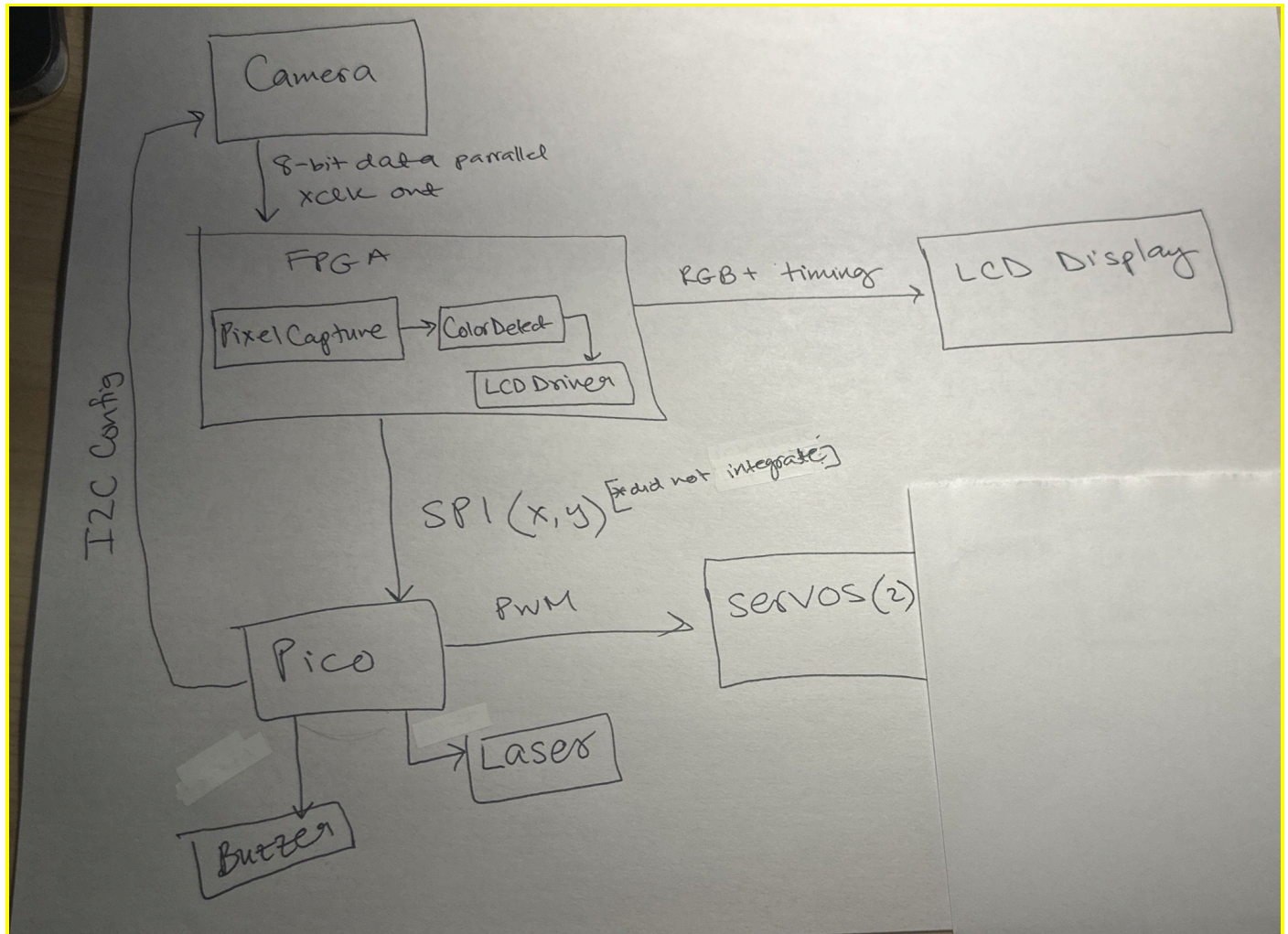
Requirements Met

- In the proposal, we attempted to achieve the idea of a working laser turret that would do color tracking with a laser, a servo, and a buzzer. This project idea got us very excited and motivated; however, when we began working on this, we faced several challenges.
- For the requirements met, we have working subsystems: a live camera view on the LCD, servo motion, and laser + buzzer functionality. Each of these works independently; when you power everything on, the camera picks up your image (blurry, with incorrect color mapping) and the servos, laser, and buzzer all operate. What's missing is the integration: the color-detection link that would tie what the camera sees to how the turret moves.
- Our unmet requirement is integrating the entire system, so that there is communication between the camera and FPGA, and pico communication to the FPGA via SPI, so that the color detection could work.

Wiring Diagram



Design Diagram



AI Usage

- Samantha:
 - How to Implement I2C on the Pico:
<https://gemini.google.com/share/f84b514ad94b>
 - How to crop the **OV7670** output to fit LCD screen:
<https://gemini.google.com/share/99605cc9e359>

- Erick:
 - How to implement the servos, laser, and buzzer
 - <https://claude.ai/share/1133e3e7-9ef2-4573-8782-91d05decbbb5>
- Laiba:
 - Understanding camera OV7670 pins and project folder setup
 - <https://claude.ai/chat/a760c7b2-92f5-4976-b4d8-6f556c7d1d66>
 - How to do camera → LCD + debugging help
 - <https://claude.ai/chat/324bc0cd-b99f-4111-ae72-553fe425184c>

Acknowledgements

1. <https://community.element14.com/members-area/personalblogs/b/ralph-ya-mamoto-s-blog/posts/camera-module-for-raspberry-pi-pico>
2. https://github.com/AngeloJacobo/FPGA_OV7670_Camera_Interface
3. youtube.com/watch?si=L_F6bF7ZO9VCPD6a&v=iW7oGRXX_dY&feature=youtu.be
4. <https://youtu.be/vVBmgjle2A0?si=Or01kN9noVgDSJDr>
5. https://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf
6. <https://www.utmel.com/components/ov7670-camera-module-datasheet-specifications-and-comparison?id=797>